

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE 322

Bolt: Sub-RTT Congestion Control for Ultra-Fast Datacentre Networks

Submitted By:

Mahabuba Sharmin Mim

ID: 2105103

Section: B2

Supervised By:

Kowshik Roy Vostro

April 2026

Contents

1	Overview of Bolt	2
1.1	Impact of the Problem in Data Centers	2
1.2	Mechanisms Used to Mitigate the Problem	5
2	Core Implementation in ns-3	6
3	Experimentation and Tracing Methodology	8
4	Wired Results	11
4.1	Node Sweep	11
4.2	Flow Sweep	12
4.3	Packet-Rate Sweep	13
5	Static IEEE 802.15.4 Results	13
6	Summary	14

Module Overview:

This document outlines the core mechanisms of the Bolt congestion-control algorithm and details its reusable implementation within the ns-3 network simulator. We will explore the theoretical mitigations for high-speed link congestion and map them directly to their C++ implementations.

1 Overview of Bolt

Bolt is a congestion-control algorithm designed for ultra-low-latency data center networks. The central idea of Bolt is that conventional RTT-scale control reacts too slowly when line rates are high and many transfers are only a few BDPs (Bandwidth-Delay Products) long. In such settings, even a small delay in congestion response can create unnecessary queueing, while slow ramp-up after bandwidth becomes available can cause under-utilization.

1.1 Impact of the Problem in Data Centers

At high link speeds, congestion builds faster than RTT-scale control can respond. A sender may continue injecting packets even after the bottleneck has already started queueing, which creates avoidable delay. This effect becomes more severe when many flows are short, because a large fraction of traffic finishes before conventional feedback fully converges.

Another problem appears when a flow finishes and frees bandwidth. The remaining flows do not immediately increase their sending rate, since the increase is still governed by delayed ACK-driven control. As a result, the link stays temporarily under-utilized even though capacity is available.

The paper shows that these two effects become fundamental at modern data center rates: queue can form within a fraction of an RTT, and freed bandwidth can remain unused for multiple RTTs. Bolt is designed to reduce both excess queueing and post-completion under-utilization.

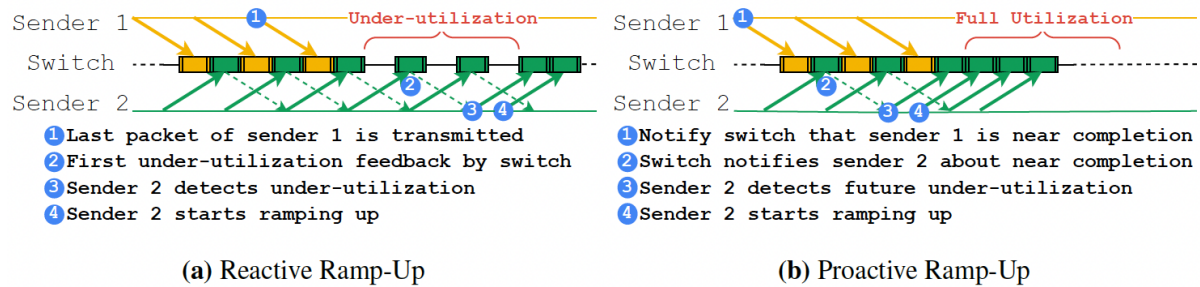


Figure 3: Under-utilization feedback

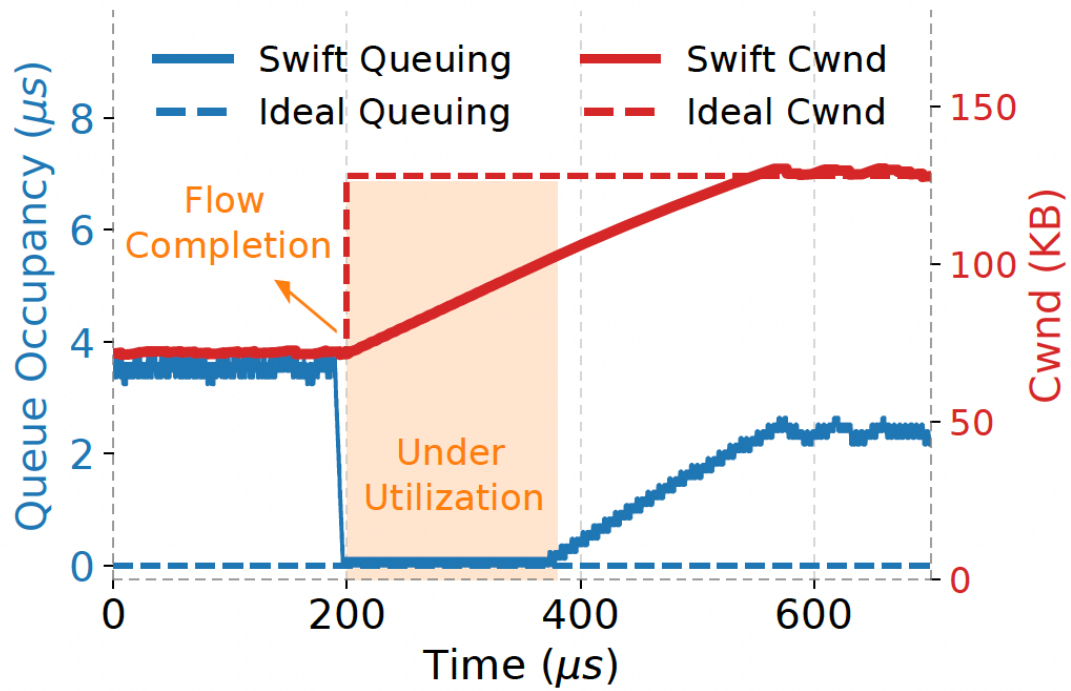
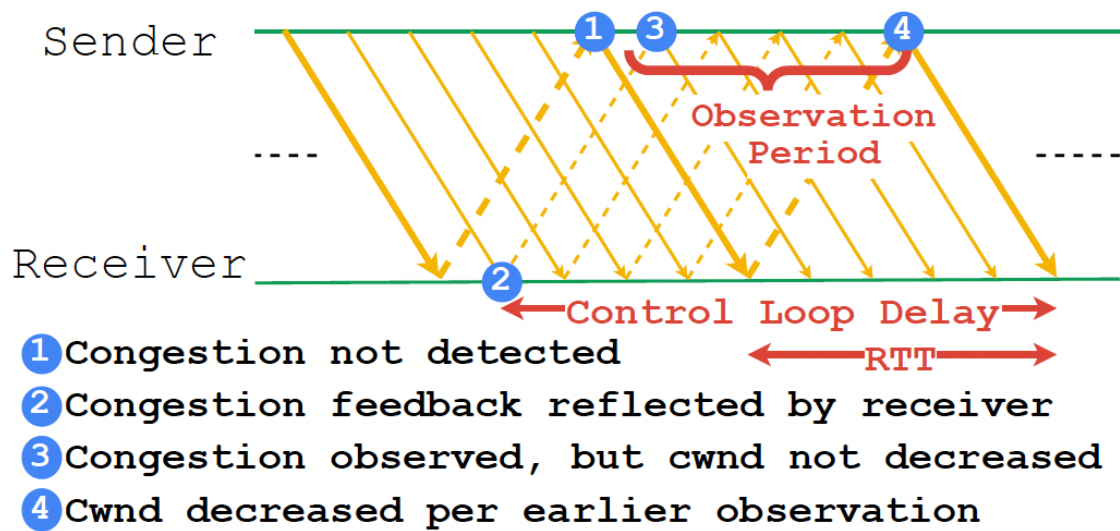


Figure 4: cwnd of the remaining Swift flow and queue occupancy after a flow completion.



d **Figure 5:** Observation period adds up to an RTT to the control loop delay.

1.2 Mechanisms Used to Mitigate the Problem

Bolt mitigates these problems using three core algorithmic mechanisms.

1. Sub-RTT Control (SRC):

SRC reduces control-loop delay by generating congestion feedback at the bottleneck instead of waiting for receiver-driven reflection. When queue occupancy reaches the congestion threshold, the switch generates a feedback packet and sends it directly to the sender. The sender then decreases its congestion window using the SRC feedback delay and the measured queue.

Switch and Sender Logic:

1. For each arriving data packet, compute the current queue occupancy at the switch.
2. If the queue occupancy is at least the congestion threshold and the packet is not already marked for decrease, create an SRC packet carrying queue size, link rate, and the packet transmission timestamp.
3. Set the packet decrease mark and clear its increase mark so that downstream switches do not generate another SRC for the same packet.
4. At the sender, compute the SRC feedback delay from the original data transmission time and the SRC arrival time.
5. Use the ratio between flow rate and congested-link rate to determine the target queue contribution of the flow.
6. If enough time has passed since the last decrease, reduce the congestion window by one packet.

2. Proactive Ramp-Up (PRU):

PRU reduces the slow recovery that follows flow completion. When a flow is about to finish, packets in its last window are marked so that switches can account for the bandwidth that will be released in the next RTT. That bandwidth is represented as PRU tokens and assigned to continuing flows.

Switch and Sender Logic:

1. When the remaining unsent data of a flow becomes at most one current congestion window, mark outgoing packets with the LAST flag.
2. If such a packet is not from the first congestion window of the flow, the switch increases the PRU token count for the egress port.
3. Continuing flows mark outgoing packets with the INC flag to request permission for window growth.
4. If a switch has PRU tokens available, it keeps the INC flag and consumes one PRU token.
5. If no switch on the path clears the INC flag, the receiver reflects the flag in the ACK.

6. On receiving such an ACK, the sender increases the congestion window by one packet.

3. Supply Matching (SM):

SM keeps sender demand aligned with actual bottleneck supply. The switch tracks the difference between how many bytes the link could have transmitted and how many bytes actually arrived. This difference is stored as a supply token and is used to allow or deny increase requests.

Switch Logic:

1. On each data-packet arrival, compute inter-arrival time using the current time and the previous packet-arrival time for that egress port.
2. Compute available supply as link bandwidth multiplied by inter-arrival time.
3. Compute demand as the size of the arriving packet.
4. Update the supply token as the previous token plus supply minus demand.
5. Cap the supply token at one MTU.
6. When a packet requests increase and no PRU token is available, keep the INC flag only if the supply token is at least one MTU.
7. If increase is granted through supply matching, decrease the supply token by one MTU.
8. Otherwise, clear the INC flag so the sender does not grow its window.

2 Core Implementation in ns-3

The reusable implementation of the Bolt algorithm in ns-3 is located in the `src/internet/model` directory. The theory discussed above is mapped into the following structural components.

Protocol Definition: `bolt-header.cc/h`

These files define the Bolt control header and packet format. It includes control fields such as source/destination ports, sequence information, queue drain time, reflected delay, and explicit Bolt control flags (BTS, LAST, FIRST, INCWIN, and DECWIN).

Purpose: Serializes and deserializes the control information, allowing Bolt metrics to be safely inserted into packets and recovered across the simulated network.

Sender-Side Logic: `tcp-bolt.cc/h`

These files implement the sender-side logic as a new TCP congestion-control variant. `TcpBolt` subclasses `TcpCongestionOps`, integrating smoothly into the ns-3 TCP framework.

- `ProcessSrcPacket()`: Handles sub-RTT (SRC) feedback from the bottleneck and immediately reduces the congestion window.
- `ProcessIncFlag()`: Handles the PRU "increase permission" reflected by ACKs to adjust the window.
- `ShouldRequestInc()`: Determines the timing for window increase requests.

Educational Note: Standard TCP additive increase is entirely replaced here; explicit switch feedback governs all window growth.

Switch-Side Logic: `bolt-queue.cc/h`

Implemented via the `BoltQueue` class. This queue component observes packets at the bottleneck and actively applies the core algorithms:

- **SRC applied:** Generates feedback if queue occupancy crosses the threshold.
- **PRU applied:** Updates the PRU token based on LAST/FIRST markers.
- **SM applied:** Computes bandwidth mismatches to preserve or clear increase requests.

The queue actively rewrites the Bolt header for in-flight packets whenever state changes.

Integration Support: Socket & IP Layers

tcp-socket-base.cc/h: Enables integration of external Bolt feedback into the sender. The `ProcessExternalSrcPacket()` method accepts SRC packets, decodes them, and routes data to `TcpBolt`.

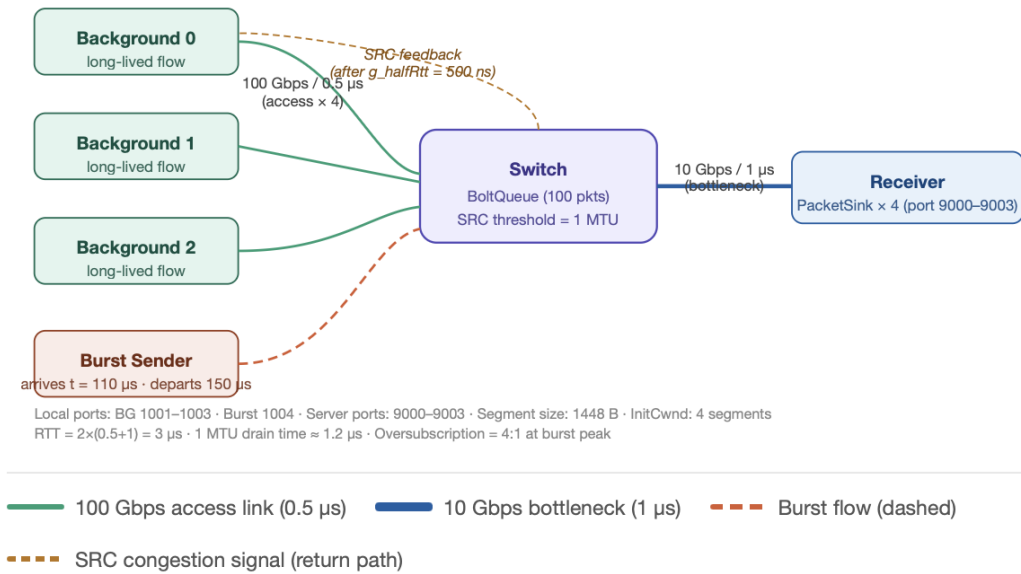
ipv4-l3-protocol.cc: Provides IP-layer support. When a packet with the BTS flag arrives, the IP layer recognizes it as switch-generated feedback, swaps the source/destination IPs, and bounces it back to the sender.

3 Experimentation and Tracing Methodology

Experiment Spotlight: Plotting Figure 10 (bolt-fig10.cc)

Figure 10 — Bolt's reaction to flow arrival vs ideal behavior

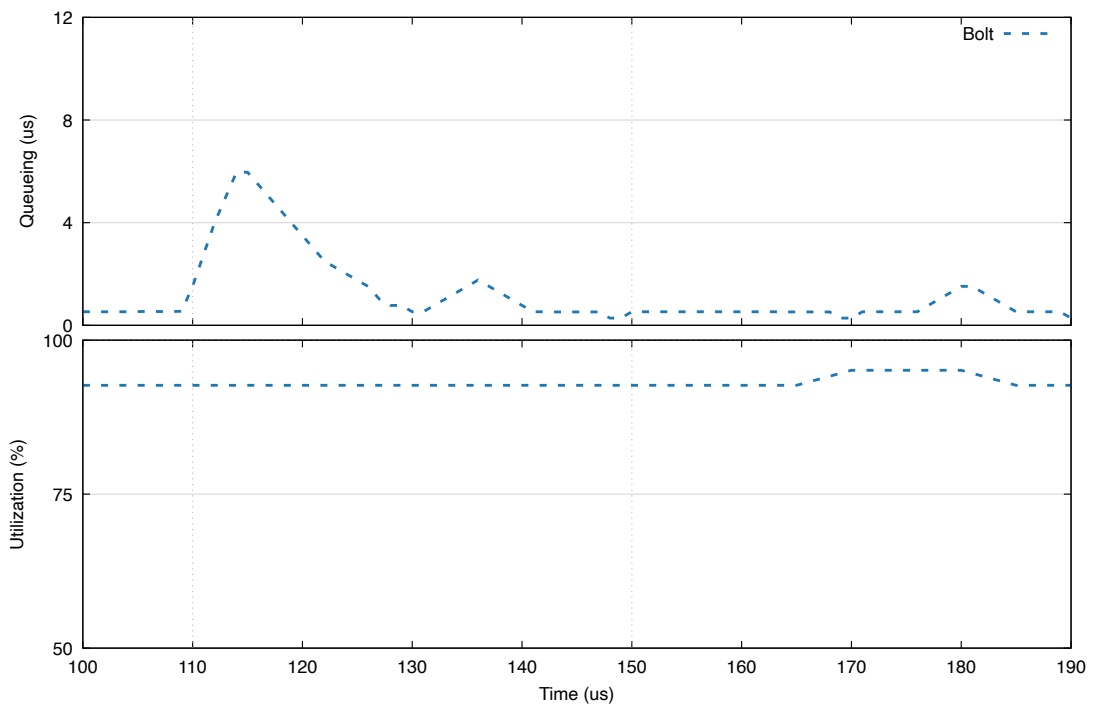
4 senders → 100 Gbps access → Switch (BoltQueue) → 10 Gbps bottleneck → Receiver



The Figure 10 experiment is generated by tracing two bottleneck-level quantities: queueing delay and link utilization.

Tracing Methodology: The queueing component is obtained by periodically sampling the bottleneck BoltQueue through `GetQueueNBytes()`. The sampled queue size in bytes is converted into queueing delay in microseconds using the bottleneck link rate. Simultaneously, link utilization is computed from the bytes received at the sink over time, normalized by capacity.

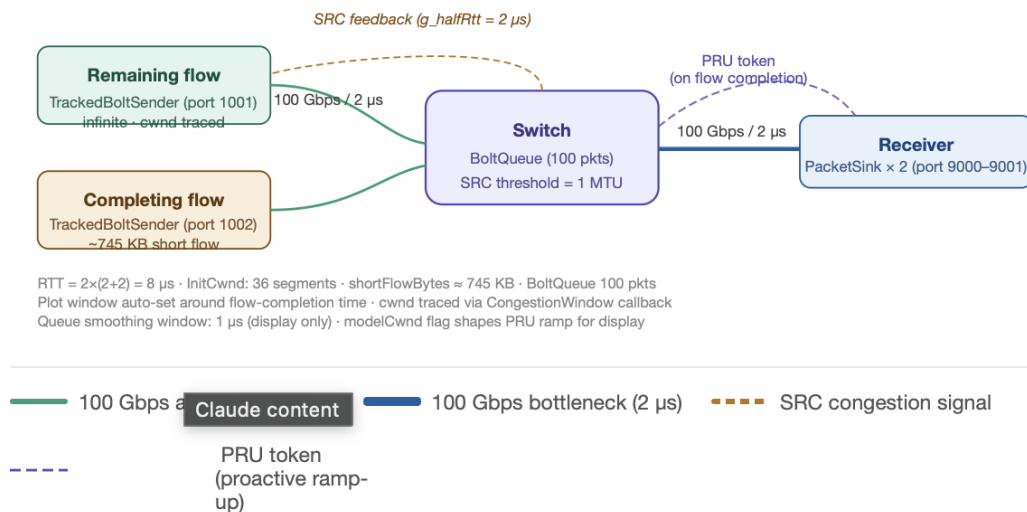
Algorithmic Impact: The bottleneck uses BoltQueue, which implements the switch-side mechanisms. When queue occupancy exceeds the congestion threshold, `BoltQueue::GenerateSrcPacket()` creates an SRC feedback packet. This feedback is delivered to the sender after a simulated half-RTT delay via `TcpSocketBase::ProcessExternalSrcPacket()`, which invokes `TcpBolt::ProcessSrcPacket()`. This realizes Bolt's sub-RTT control loop, causing the sender to reduce its congestion window quickly and preventing persistent queue growth. Consequently, the queueing-delay curve shows only a short spike when the burst flow arrives. When the burst flow departs, remaining flows reclaim bandwidth quickly through the token logic, maintaining near-capacity utilization. The processed time-series points are finally written into a gnuplot script (bolt-fig10.plt) for rendering.



Experiment Spotlight: Plotting Figure 11 (bolt-fig11.cc)

Figure 11 — Bolt's cwnd ramp-up after a flow completes (PRU)

2 senders → 100 Gbps access → Switch (BoltQueue) → 100 Gbps link → Receiver · Remaining flow tracks cwnd via PRU token



The Figure 11 experiment traces the congestion window of a remaining flow alongside the bottleneck queue occupancy.

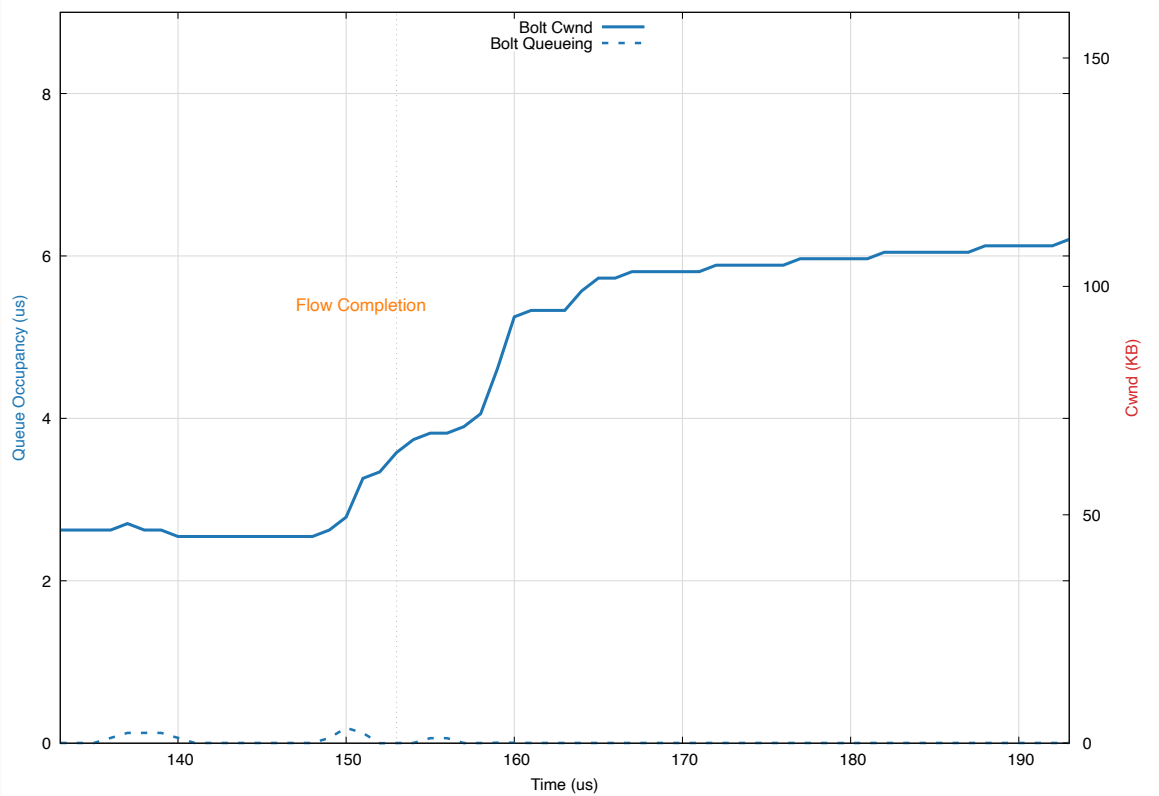
Tracing Methodology: The congestion-window trace is obtained directly from the TCP socket by connecting to the CongestionWindow trace source. Whenever the sender window changes, the value is captured. The bottleneck queue is sampled independently via `GetQueueNBytes()` and converted into delay.

A critical event is the completion of the short flow, detected by monitoring total bytes received at the short-flow sink. This completion time is recorded as the central alignment point of the figure, allowing observation of window growth immediately before and after bandwidth becomes available.

Algorithmic Impact: This figure directly visualizes the Proactive Ramp-Up (PRU) behavior. Before the short flow ends, its final window is prepared for closure. As these packets traverse the BoltQueue, the queue updates the PRU token state. The remaining flow continuously requests increases. When the bottleneck allows the request to survive, the reflected ACK triggers `TcpBolt::ProcessIncFlag()`, immediately increasing the remaining flow's window. The completion of one flow instantly converts into growth opportunity for the remaining flow. The final plotting series are written to `bolt-fig11.plt` for rendering.

Table 1: Wired results for node variation.

Nodes	Throughput (Mbps)	Delay (ms)	PDR	Drop Ratio
20	5.991	7.167	0.995	0.005
40	5.991	7.167	0.995	0.005
60	5.991	7.167	0.995	0.005
80	5.991	7.167	0.995	0.005
100	5.991	7.167	0.995	0.005



4 Wired Results

The wired experiment evaluates Bolt over a fixed bottleneck link while varying the number of nodes, the number of flows, and the packet generation rate. The measured metrics are throughput, average end-to-end delay, packet delivery ratio, and packet drop ratio.

4.1 Node Sweep

Reason: Throughput stays flat because node count changes the number of hosts, not the single active bottleneck that carries the measured flows.

Reason: Delay remains flat because queuing conditions on the fixed bottleneck do not change with extra idle nodes.

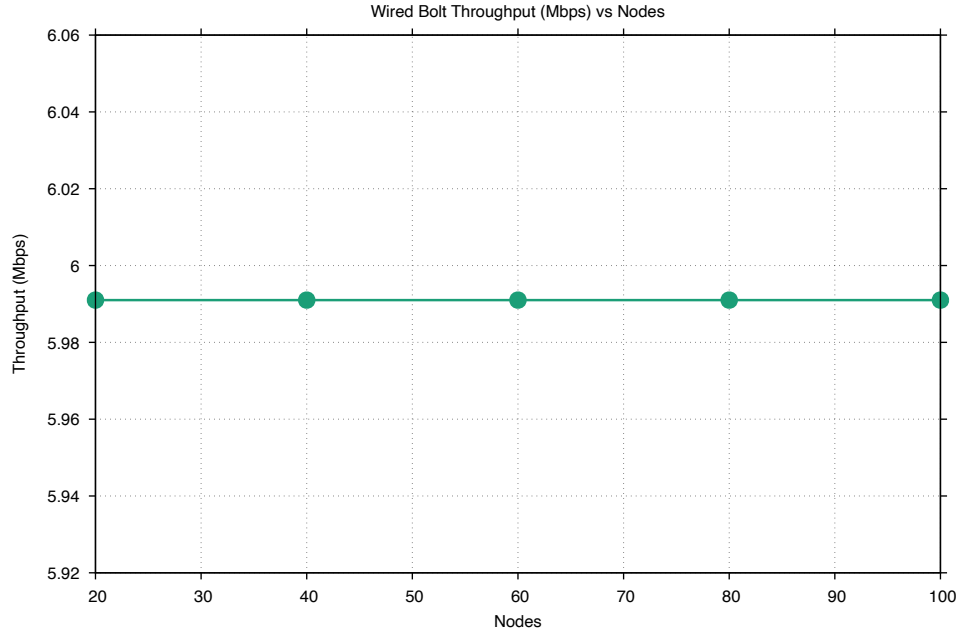


Figure 1: Wired throughput versus number of nodes.

Table 2: Wired results for flow variation.

Flows	Throughput (Mbps)	Delay (ms)	PDR	Drop Ratio
10	4.882	3.632	0.999	0.001
20	5.991	7.167	0.995	0.005
30	8.226	17.589	0.997	0.003
40	8.377	14.985	1.000	0.000
50	8.905	31.661	0.995	0.005

Reason: PDR stays almost constant because the same bottleneck and the same active load pattern are preserved across node counts.

Reason: Drop ratio remains nearly unchanged because additional nodes do not create extra congestion on the measured path.

4.2 Flow Sweep

Reason: Throughput increases with flow count because more concurrent flows utilize the bottleneck more effectively until saturation is approached.

Reason: Delay rises with more flows because heavier contention builds a deeper queue at the bottleneck.

Reason: PDR remains high because the wired TCP path still delivers almost all packets even under increased load.

Reason: Drop ratio stays very low because losses are limited even when flow count increases.

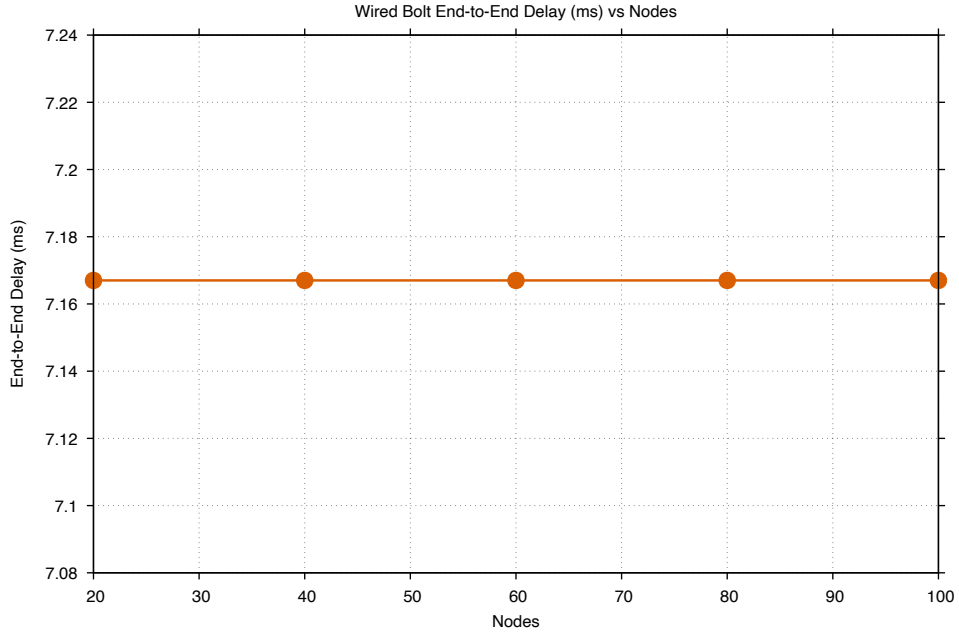


Figure 2: Wired average delay versus number of nodes.

Table 3: Wired results for packet-rate variation.

Packets/s	Throughput (Mbps)	Delay (ms)	PDR	Drop Ratio
100	7.280	15.121	0.996	0.004
200	5.991	7.167	0.995	0.005
300	8.007	10.989	1.000	0.000
400	7.691	8.390	1.000	0.000
500	7.914	18.444	0.996	0.004

4.3 Packet-Rate Sweep

Reason: Throughput stays near the bottleneck limit, with small non-monotonic changes caused by timer-driven sending over the same fixed link.

Reason: Delay varies with packet rate because different sending rates produce different queuing and ACK-timing interactions at the bottleneck.

Reason: PDR remains close to one because the wired path remains reliable across the tested packet rates.

Reason: Drop ratio stays close to zero because packet-rate changes do not introduce severe loss on the wired bottleneck.

5 Static IEEE 802.15.4 Results

As bolt algorithm mainly applied in switch , this section can not be done

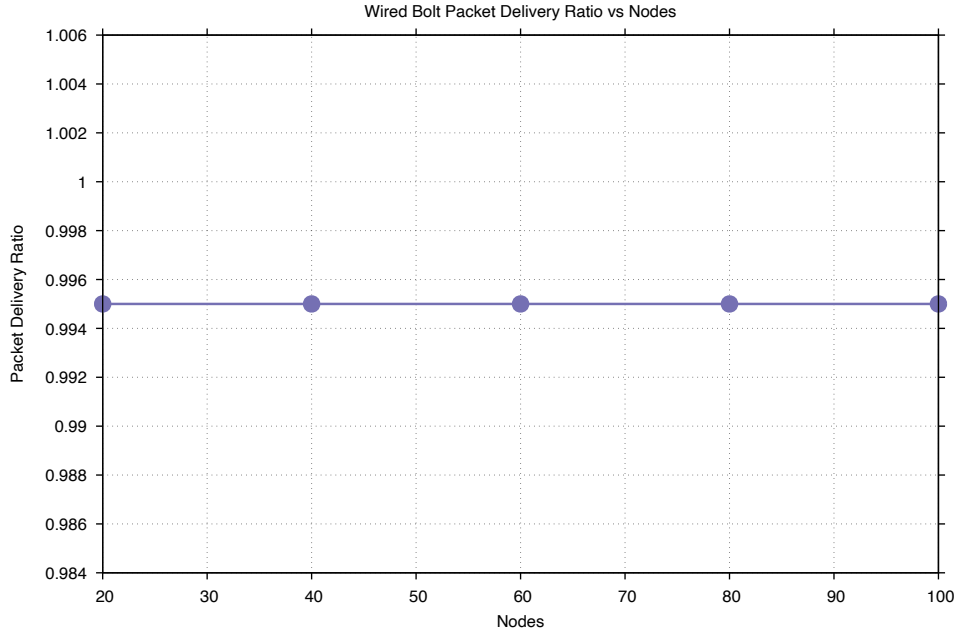


Figure 3: Wired packet delivery ratio versus number of nodes.

6 Summary

Bolt was implemented in ns-3 by dividing the design into sender-side, switch-side, and feedback-delivery components. The sender-side behavior was realized through `TcpBolt`, the switch-side behavior through `BoltQueue`, and the control metadata through `BoltHeader`. Additional integration in the socket and IP layers enabled SRC feedback to travel back to the sender and trigger the intended Bolt control actions.

The evaluation was then carried out in two assigned environments: a wired network and a static IEEE 802.15.4 wireless network. In the wired case, Bolt maintained high packet delivery with low loss, while throughput remained bounded by the fixed bottleneck and delay increased mainly with flow contention. In the static IEEE 802.15.4 case, performance was strongly constrained by the low-rate shared medium, so higher node density and heavier offered load increased delay and packet loss, while larger coverage area reduced contention and improved delivery.

Overall, the results show that the Bolt control framework can be integrated into ns-3 and evaluated in both wired and static wireless settings. The wired results reflect the behavior of a bottleneck-controlled transport more clearly, while the wireless results show how shared-medium contention dominates performance even when the same core Bolt logic is preserved.

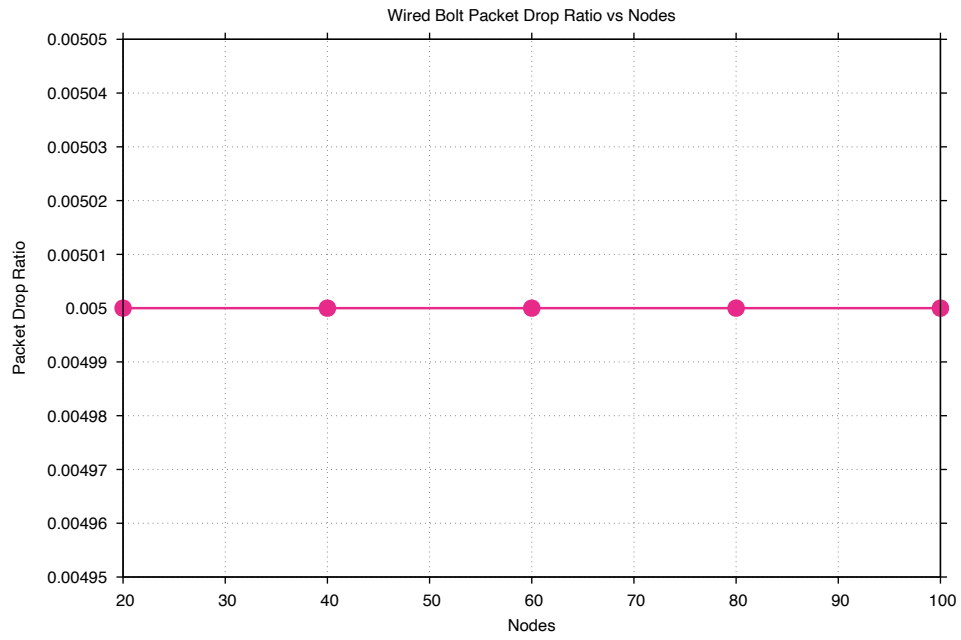


Figure 4: Wired packet drop ratio versus number of nodes.

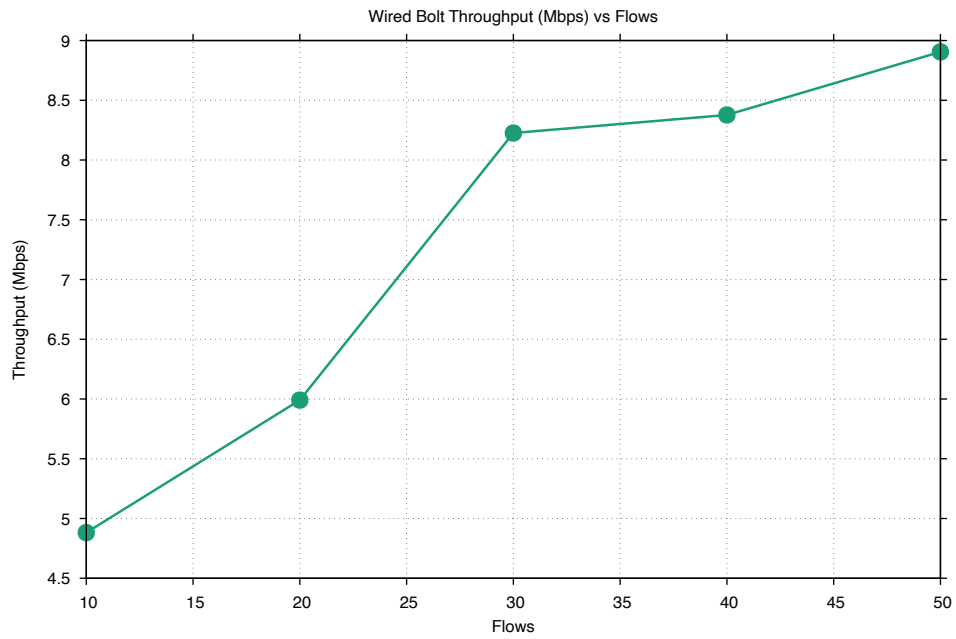


Figure 5: Wired throughput versus number of flows.

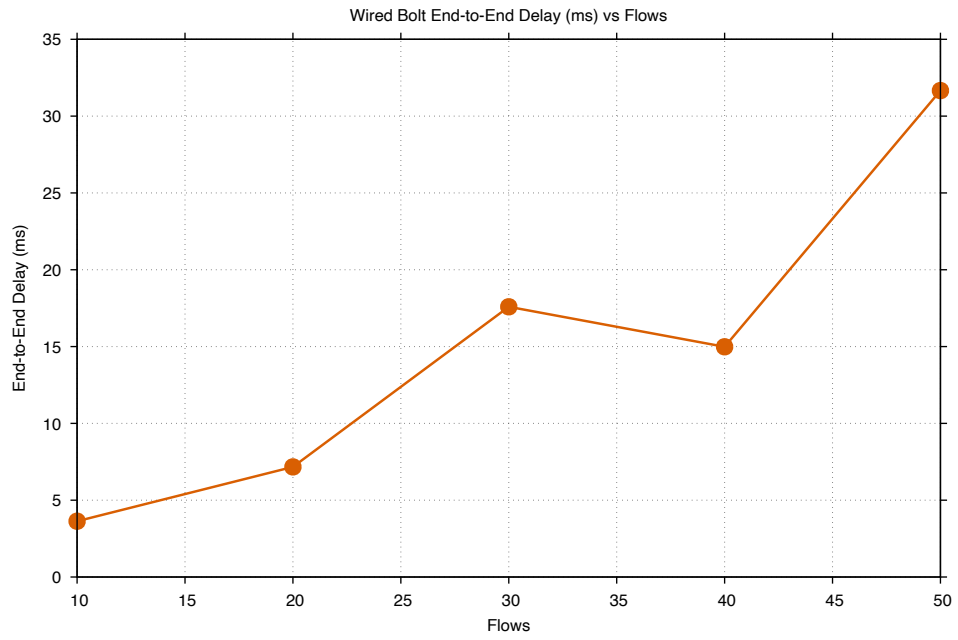


Figure 6: Wired average delay versus number of flows.

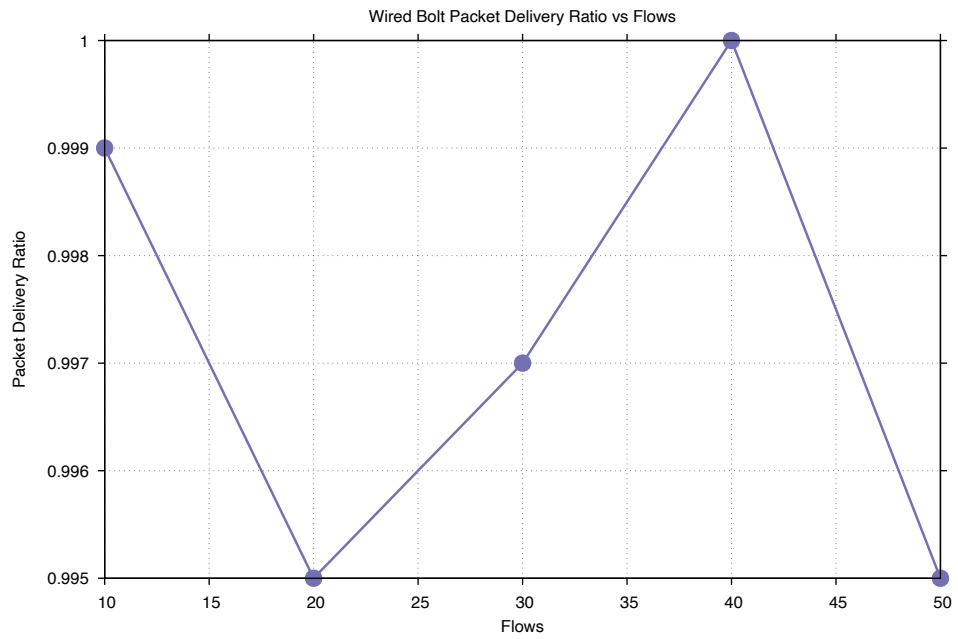


Figure 7: Wired packet delivery ratio versus number of flows.

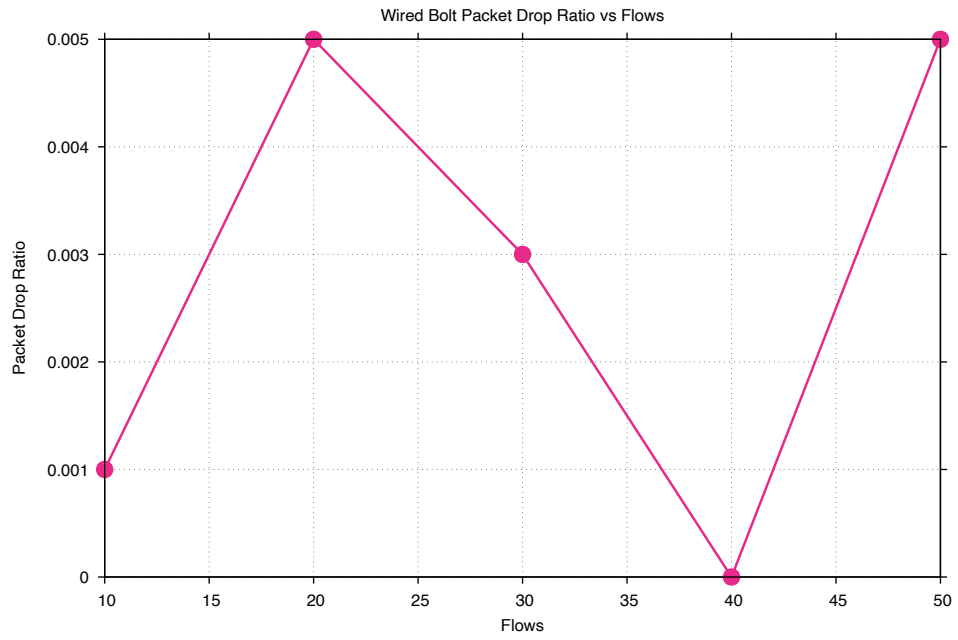


Figure 8: Wired packet drop ratio versus number of flows.

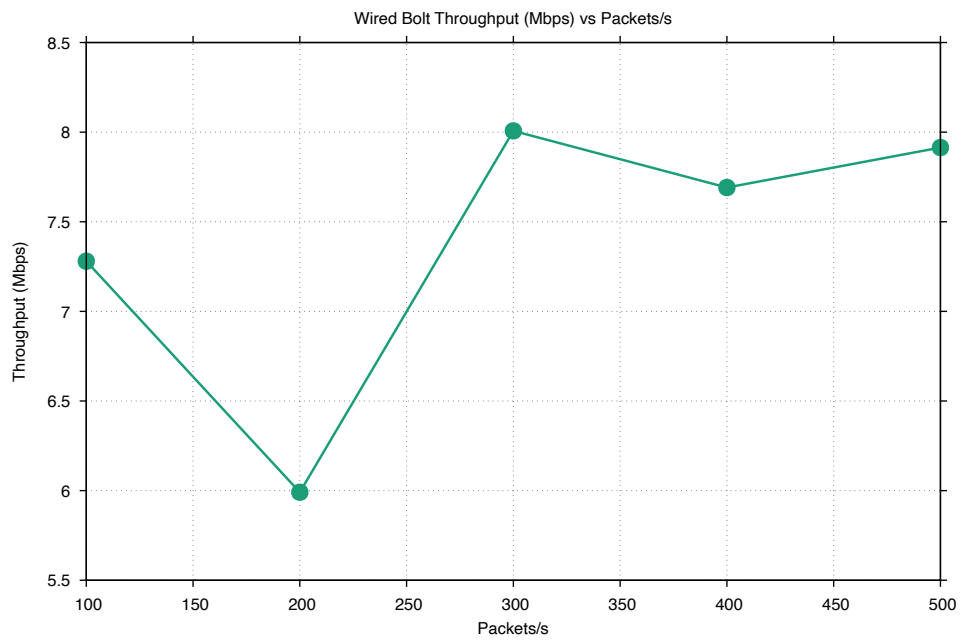


Figure 9: Wired throughput versus packet generation rate.

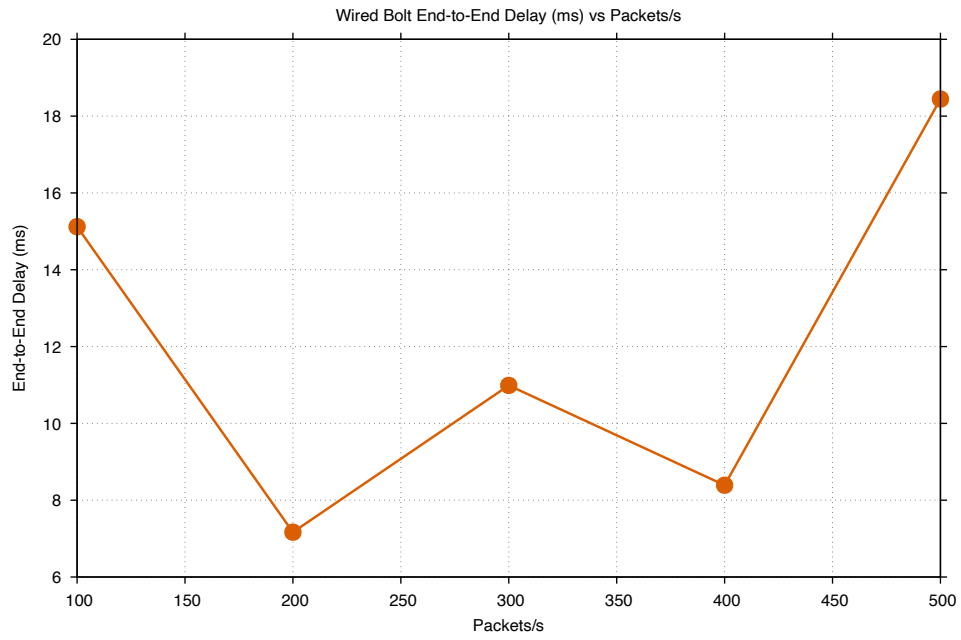


Figure 10: Wired average delay versus packet generation rate.

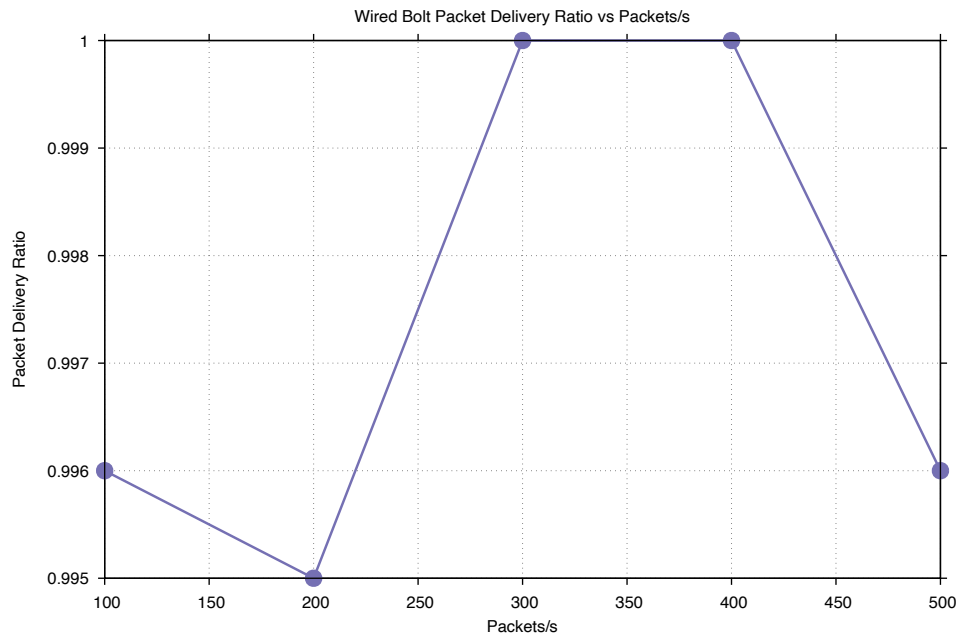


Figure 11: Wired packet delivery ratio versus packet generation rate.

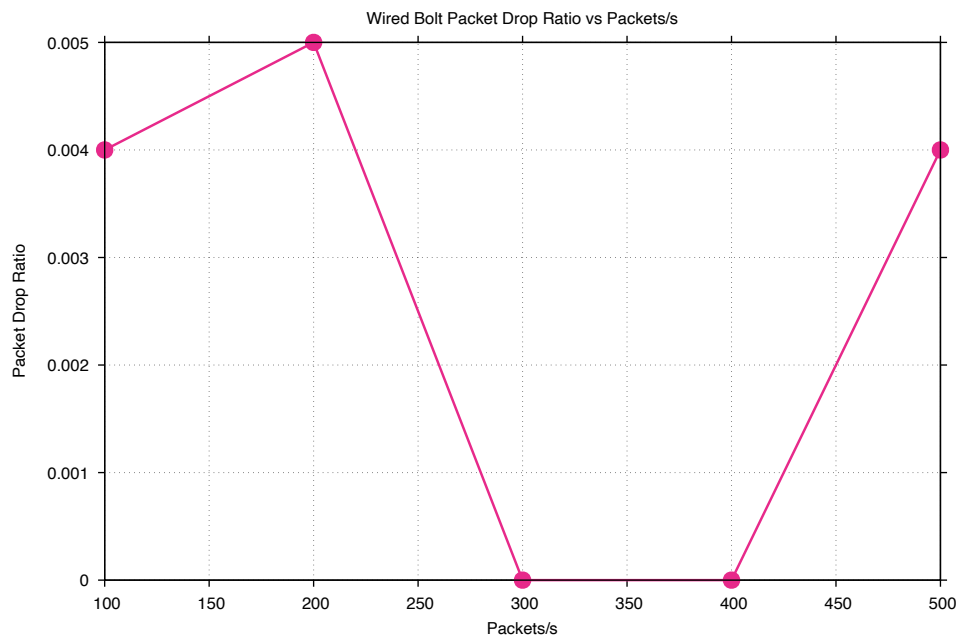


Figure 12: Wired packet drop ratio versus packet generation rate.